

Desarrollo de Sistemas Móviles

Sesión 8: : Frameworks Backend para Apps Móviles

Spring Boot · Ktor · Quarkus · FastAPI · Microservicios · OpenAPI

<https://platform-api.kankunapaq.com/>

Al finalizar la clase, serás capaz de:

- Comparar frameworks backend 2026: Spring Boot, Ktor, Quarkus, FastAPI; cuándo usar cada uno.
- Entender Spring Boot 3.3+ / 4.0: Virtual Threads (Project Loom), GraalVM native image y Spring Security 6.2.
- Aplicar Ktor 2.3+ / 3.x: DSL de routing con coroutines nativas, configuración sin anotaciones.
- Entender microservicios: diferencias con el monolítico, API Gateway y Event-Driven Architecture con Kafka.
- Diseñar API-First con OpenAPI 3.1: el equipo Android genera el cliente Retrofit antes de que el backend esté listo.

- La app móvil es tan buena como su backend: la latencia, disponibilidad y escalabilidad vienen del servidor.
- El equipo Android/Kotlin necesita saber qué esperar del backend para diseñar la capa de red (Retrofit, auth, errores).
- En proyectos reales el mismo equipo toca frontend móvil y backend; hoy cubrimos el lado servidor.
- Semana pasada: ORM/DAO (cómo guardar datos).
- Hoy: el framework que expone estos datos como API REST.

App Android / iOS

Retrofit · Auth · Manejo de errores



Backend / API REST

Latencia · Disponibilidad · Escalabilidad

Del ORM/DAO al API REST

Semana 7 cubrió cómo guardar datos (ORM/DAO). Semana 8 cubre el framework que expone esos datos como API REST consumible por la app.

Comparativa de frameworks backend 2026

Framework	Lenguaje	Startup (JVM / Native)	RAM Docker	Cuándo usarlo
Spring Boot 4.0 (v 3.3+)	Java / Kotlin	~2s / 50ms	200MB / 50MB	Enterprise, ecosistema maduro
Quarkus 3.8+	Java / Kotlin	~300ms / 50ms	120MB / 30MB	Cloud-native, Kubernetes-first
Ktor 3.4+	Kotlin	~200ms	80MB	Equipos Kotlin, APIs ligeras
FastAPI 0.115+	Python	~500ms	100MB	Equipos Python, backends IA/ML
Go Echo 4.12+	Go	~10ms	15MB	Alta concurrencia, microservicios

Actualización 2026: Spring Boot 3.5 llegó a EOL open-source el 30 de junio de 2026; Spring Boot 4.0 (sobre Spring Framework 7 / Jakarta EE 11) es hoy la recomendación para proyectos nuevos. Ktor avanzó a la serie 3.x (3.5.0, mayo 2026), con generación de documentación OpenAPI integrada.

¿Cómo elegir el framework correcto?

Spring Boot 4.0

Equipo Java/Kotlin, integración empresarial, ecosistema maduro.

Quarkus 3.8+

Proyecto en Kubernetes, startup < 1 segundo requerido.

Ktor 3.x

Equipo 100% Kotlin, control total sobre la configuración, sin "magia".

FastAPI

Equipo Python, el backend tiene componentes de IA/ML.

Go Echo

Máxima concurrencia con mínima RAM (microservicios de alta carga).

Regla de oro

Elegir por el equipo y el contexto del proyecto, no solo por benchmarks de rendimiento.

Los benchmarks importan, pero la productividad del equipo y la curva de aprendizaje deciden el éxito a largo plazo.

Framework más usado a nivel global (Stack Overflow Developer Survey 2024).

Basado en convención sobre configuración: starters, auto-configuración, application.properties. Java 21 (LTS) es la base recomendada para aprovechar Virtual Threads, Records y Sealed Classes.



Virtual Threads (Project Loom)

Escalar sin escribir código reactivo.



GraalVM Native Image

Compila a binario nativo → startup ~50ms.



Spring Security 6.2+

Configuración DSL, OAuth2/JWT de primera clase.

```
application.properties

spring.threads.virtual.enabled=true

@GetMapping("/productos")
fun getProductos(): List<Producto> {
    Thread.sleep(200)
    return repository.findAll()
}
```

*Código síncrono/bloqueante → rendimiento asíncrono, sin
async/await ni Flows.*

2026: Spring Boot 4.0 (Spring Framework 7, Jakarta EE 11) sucede a la línea 3.x, que llegó a EOL open-source el 30 jun. 2026.

Problema tradicional: un hilo del pool bloqueado esperando I/O no puede atender otras peticiones.

~200

hilos del pool tradicional,
simultáneos

Millones

de hilos virtuales, gestionados por
la JVM

Escribe código síncrono/bloqueante y obtienes rendimiento asíncrono; sin async/await ni Flows.

2026: En Spring Boot 4.0, los clientes HTTP basados en JDK HttpClient también usan Virtual Threads automáticamente cuando: `spring.threads.virtual.enabled = true`, extendiendo el beneficio a las llamadas salientes.

Producto.kt

```
@GetMapping("/productos")
fun getProductos(): List<Producto>
{
    Thread.sleep(200) // I/O
    return repository.findAll()
}
```

El virtual thread se suspende sin bloquear el hilo de plataforma subyacente.

config	SecurityConfig, CorsConfig, SwaggerConfig
controller	@RestController: solo routing y mapeo HTTP
service	Lógica de negocio (≈ UseCase en Android)
repository	Spring Data JPA interfaces
entity	@Entity: clases que mapean tablas de BD
dto	Data Transfer Objects (request / response)
mapper	Conversiones Entity ↔ DTO
exception	Excepciones custom + @ControllerAdvice

Controller

solo HTTP: recibe request, delega, responde



Service

no sabe nada de HTTP: solo lógica de negocio



Repository

Repository


```
SecurityConfig.kt

@Bean
fun securityFilterChain(http: HttpSecurity) {
    http
        .csrf { it.disable() }
        .authorizeHttpRequests {
            it.requestMatchers("/api/public/**")
                .permitAll()
            it.requestMatchers("/api/v1/**")
                .authenticated()
            it.requestMatchers("/admin/**")
                .hasRole("ADMIN")
        }
        .oauth2ResourceServer { it.jwt { } }
    return http.build()
}
```

/api/public/**

Sin autenticación (login, registro, health check)

/api/v1/**

Requiere JWT válido en header Authorization: Bearer <token>

/admin/**

Requiere JWT + rol ADMIN

*El JWT lo emite Firebase Auth o Auth0
Spring solo lo valida.*

- Desarrollado por JetBrains, usa coroutines nativas de Kotlin.
- Configuración por DSL, sin anotaciones (@RestController, @Autowired no existen).
- Menor "magia" que Spring: lo que no configuras explícitamente, no existe.
- Se instalan plugins (equivalente a starters): ContentNegotiation, Authentication, CORS, CallLogging.

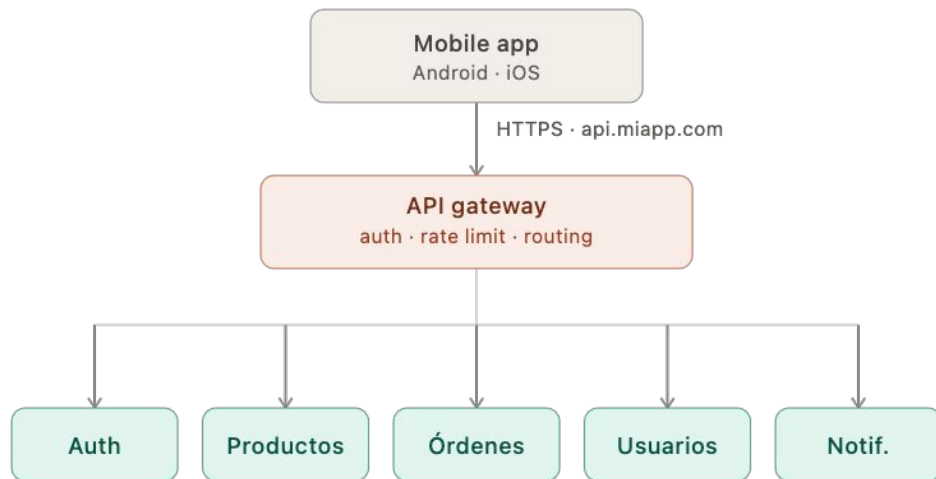
2026: Ktor avanzó a la serie 3.x. La versión 3.4.0 (ene. 2026) incorporó generación de documentación OpenAPI integrada, y 3.5.0 (may. 2026) continuó mejorando servidor y cliente.

```
Application.kt

embeddedServer(Netty, port = 8080) {
    install(ContentNegotiation) { json() }
    install(Authentication) {
        jwt("auth-jwt") { /* ... */ }
    }
    routing {
        get("/api/v1/productos") {
            call.respond(service.getAll())
        }
        post("/api/v1/productos") {
            val req =
call.receive<CreateProductoRequest>()
            call.respond(HttpStatus.Created,
                service.create(req))
        }
    }
}.start(wait = true)
```

	Monolítico	Microservicios
Estructura	1 backend, 1 base de datos	N servicios independientes, cada uno con su BD
Desarrollo inicial	Simple	Complejo (red, contratos, orquestación)
Escalado	Todo o nada	Por servicio (solo el de órdenes si hay pico)
Fallos	Un bug puede tumbar todo	Fallos aislados por servicio
Testing local	Fácil	Difícil (múltiples contenedores)
Cuándo usarlo	MVP, equipos pequeños, primer año	Alta escala, equipos grandes, dominios definidos

Regla de oro: empezar monolítico, migrar a microservicios cuando el monolito duele.



Un solo punto de entrada — los microservicios no se exponen directamente

- Autenticación centralizada: el gateway valida el JWT, los servicios no necesitan saber de auth.
- Rate limiting: protección contra abuso de la API desde la app.
- Opciones: Kong, AWS API Gateway, Nginx, Spring Cloud Gateway.

La app móvil siempre habla con un solo punto de entrada, no con cada microservicio directamente.

En lugar de llamadas síncronas entre servicios, se publican eventos.



Si Inventario cae, Kafka conserva el evento hasta ser procesado

Órdenes → publica

```
kafkaTemplate.send("orden-creada",  
    OrdenCreadaEvent(  
        ordenId = orden.id,  
        usuarioId = orden.usuarioId,  
        productos = orden.productos  
    ))
```

Inventario → escucha

```
@KafkaListener(topics = ["orden-creada"])  
fun handleOrdenCreada(event: OrdenCreadaEvent) {  
    event.productos.forEach {  
        inventarioService.descontarStock(  
            it.productoId, it.cantidad) }  
}
```

```
openapi.yaml

openapi: 3.1.0
paths:
  /api/v1/productos:
    get:
      summary: Obtener lista de productos
      responses:
        '200':
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Producto'
      components:
        schemas:
          Producto:
            required: [id, nombre, precio]
            properties:
              id: { type: integer }
              nombre: { type: string }
              precio: { type: number, minimum: 0 }
```

El equipo Android genera el cliente Retrofit automáticamente desde el YAML

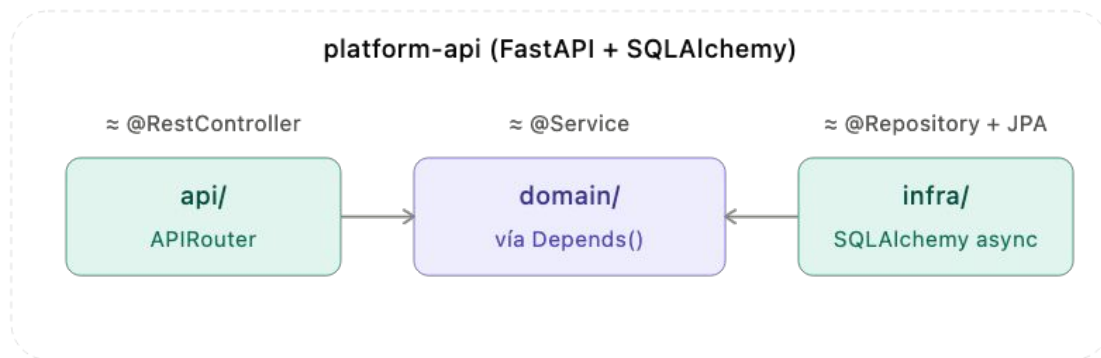
```
openapi-generator-cli generate -i openapi.yaml -g kotlin -o ./app/src/main/kotlin/api
```

Frontend y backend trabajan en paralelo desde el día 1.

Contrato como fuente de verdad

- Cambios en el YAML → regenerar cliente → errores de compilación si algo rompe el contrato.
- Spring Boot genera /docs (Swagger UI) automáticamente desde las anotaciones del controller.
- Ktor 3.4+ ahora también genera documentación OpenAPI de forma integrada.

API-First: definir el contrato (YAML) antes de escribir código.



Mismos patrones que Spring Boot: Repository, Ports & Adapters, DTO, inyección de dependencias

Concepto Spring Boot	Equivalente platform-api (FastAPI)
@RestController	APIRouter + @router.get/post
@Service	Clase Service inyectada vía Depends()
@Repository + JPA	GeoEventORMRepository + SQLAlchemy async
application.properties	credentials/layout_example.env
@Autowired	Depends(get_geo_event_orm_repo)
Swagger UI en /swagger-ui	Swagger UI en /docs (automático)

Tópicos visto

- **Spring Boot 4.0:** Virtual Threads + GraalVM = rendimiento sin código reactivo.
- **Ktor 3.x:** Kotlin puro, DSL sin anotaciones, control total.
- **Quarkus:** Cloud-native, ideal para Kubernetes.
- **FastAPI:** Python async, equivalente ligero a Spring Boot.
- **Microservicios:** Escalan por separado; empezar monolítico y migrar cuando duela.
- **API Gateway:** Único punto de entrada: auth, rate limiting, routing.
- **Kafka:** Servicios desacoplados vía eventos, resilientes ante caídas.
- **OpenAPI 3.1:** Contrato primero → cliente Retrofit generado automáticamente.

Lab

Laboratorio, Semana 10 (14)
12:15 pm

Gracias

